Agentic Deep Research System

Report on Agentic RAG for Academic Research

Suman Kar | github.com/Karsuman4298/AGENTIC-DEEP-RESEARCH | June 2026

Title: We present an end-to-end agentic retrieval-augmented generation (RAG) system for answering complex research questions over a corpus of approximately 700 arXiv papers in machine learning and NLP (2024–2026). The pipeline comprises five sequential stages—*planning, hybrid retrieval, reflective evidence assessment, grounded synthesis,* and *lexical citation verification*—evaluated across seven ablation configurations on 30 questions spanning factoid, comparative, and survey types. Our central finding is deliberately mixed: *agentic scaffolding does not uniformly outperform a strong single-pass baseline.* The baseline achieves the highest accuracy (3.77) at less than half the latency of the full agent (18.7 s vs. 30.3 s), though the full agent achieves superior coherence (4.50 vs. 4.27) and completeness (3.23 vs. 3.03). The reranker is the most impactful component for accuracy; its removal causes the largest drop (3.63). The citation verifier shows no accuracy benefit despite achieving perfect faithfulness scores. We document three concrete failure modes with root-cause analyses and propose targeted improvements.

1. Introduction

Research on LLM agents has grown at an extraordinary pace, making manual literature survey impractical. Retrieval-augmented generation (RAG) provides a natural framework: index a corpus, retrieve relevant passages, and condition an LLM to produce cited answers. Agentic extensions add iterative refinement and question decomposition, motivated by the observation that single-pass retrieval often misses multi-faceted evidence.

However, the empirical question of *whether* these additions are necessary for a bounded academic corpus remains unanswered. It is plausible that a well-engineered index with strong hybrid retrieval suffices, and that agentic overhead introduces latency and failure modes without commensurate accuracy gains. This report investigates that question directly.

Contributions:

- A reproducible corpus construction pipeline with section-aware chunking and a persistent hybrid index (BM25 + ChromaDB).
- A modular five-stage agentic RAG system with individually toggleable components and a unified multi-provider LLM client.
- A structured ablation study across seven configurations and 30 evaluation questions with empirical results.
- An honest interpretation of results, documented failure modes, and concrete improvement proposals.

2. Corpus Construction

2.1. Data Collection

Papers were collected via the arXiv public API (`collect.py`) targeting categories `cs.CL`, `cs.AI`, and `cs.LG` (Jan 2024–Apr 2026). A keyword filter on titles and abstracts retains agent-relevant papers (e.g., *agent, multi-agent, tool use, ReAct, self-reflection, planning, memory, RAG*). Each record stores arXiv ID, title, authors, abstract, category, and PDF URL in `data/raw/metadata.jsonl`. Collection is incremental—existing IDs are skipped on re-runs—yielding ≈700 papers. The collector implements polite rate-limiting with retry and exponential backoff to comply with arXiv's API terms of service.

Why arXiv? arXiv provides open, structured access to preprints with stable IDs, enabling reproducible corpus snapshots. The three selected categories cover the full breadth of LLM agent research while excluding unrelated sub-fields. Flat JSONL storage avoids database dependencies and supports simple downstream processing.

2.2. PDF Processing and Chunking

Text extraction uses **PyMuPDF** (`fitz`) via `chunk.py`. PyMuPDF was selected over alternatives such as `pdfminer` or `pypdf` because it reliably handles complex academic layouts including multi-column text, mathematical notation, and embedded figures, while being significantly faster for batch processing. Cleaning includes de-hyphenation, blank-line collapse, whitespace normalisation, and page-number removal. Chunking proceeds in four steps:

1. Detect section boundaries via regex matching standard academic headings (Abstract, Introduction, Method, Results, Conclusion).
2. Apply a sliding window of **1,800 chars** with **200 char overlap**, breaking at sentence boundaries.
3. Discard chunks shorter than 80 characters.
4. Prefix each chunk with provenance metadata:

Listing 1: Chunk provenance prefix.

```
[DOCUMENT: <title> (arXiv: <id>)]
[SECTION:  <section_name>]
<chunk_text ...>
```

Why these chunking parameters? The 1,800-character window (≈450 tokens) fits comfortably within

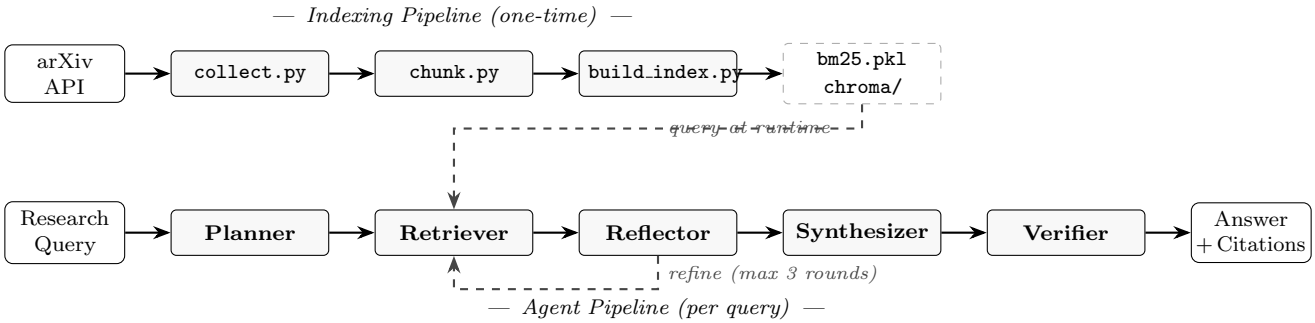

Figure 1: System architecture. *Top row:* one-time indexing pipeline (`collect.py` → `chunk.py` → `build_index.py`) producing BM25 and ChromaDB artifacts from raw PDFs. *Bottom row:* per-query five-stage agent pipeline (`Agent.py`). Dark dashed arrow (Reflector → Retriever): iterative evidence-refinement loop, up to 3 rounds. Dark dashed arrow (index store → Retriever): both indexes are queried at runtime for every sub-question.

the context budget of smaller embedding models while retaining enough surrounding text to resolve coreference within a passage. The 200-character overlap prevents evidence from being severed at chunk boundaries. Section-aware splitting preserves discourse structure so that the retriever can distinguish a claim in the Introduction from one in the Experiments section. Provenance prefixing is essential for correct `[arxiv:ID]` citation assignment and end-to-end verifier tracing without any additional database lookup.

3. System Architecture

3.1. Overview

The system comprises two sub-pipelines: a *one-time* indexing pipeline and a *per-query* agent pipeline, illustrated in Figure 1. The indexing pipeline builds persistent search infrastructure from the collected corpus; the agent pipeline is invoked fresh for every research query.

3.2. Index Construction (`build_index.py`)

Two parallel indexes are built from the chunk JSONL files in `data/chunks/`:

- **BM25** (`BM25Okapi`, serialized to `data/index/bm25.pkl`). BM25 was chosen because it provides deterministic, GPU-free lexical matching with strong recall for exact technical terms, model names, and author-coined acronyms that dense embeddings tend to conflate semantically.
- **Dense index** (`BAAI/bge-small-en-v1.5`, 384-dim cosine space, persisted in `data/index/chroma/`). `bge-small-en-v1.5` was selected because it ranks at or near the top of the BEIR benchmark for its size class (130 MB), runs on CPU within acceptable latency, and produces embeddings well-suited to passage-level retrieval in scientific text.

Why ChromaDB? ChromaDB was chosen over alternatives such as FAISS or Pinecone because it provides an embedded, serverless architecture requiring no separate process or cloud subscription. For a corpus of ∼700 papers it delivers sufficient throughput, and its Python-native API supports incremental upserts—existing chunk IDs are checked before re-embedding, making the indexing step safe to re-run after corpus additions.

3.3. Stage 1: Planner

The Planner (`Agent.py`) converts the input research question into 2–4 targeted keyword-style sub-queries via a structured LLM call (`call_llm_json()`). For short or definitional questions it falls back to rule-based keyword extraction; on any JSON parse failure it passes the original question verbatim, so the pipeline never blocks on a planning error. Sub-queries are capped at four and deduplicated before retrieval.

Why a planner? Broad research questions such as "What are recent advances in multi-agent reasoning?" contain multiple implicit sub-questions. A single retrieval query for the full question tends to over-retrieve on the most salient keyword and miss peripheral but relevant aspects. Query decomposition distributes retrieval effort across facets of the question, improving coverage for survey-type queries.

3.4. Stage 2: Retriever (`retriever.py`)

Implements hybrid search with optional cross-encoder reranking.

BM25 retrieval. Tokenise query; score all indexed chunks via `BM25Okapi`; apply hard cutoff $\tau_{\text{BM25}} = 1.0$; rank by descending score.

Semantic retrieval. Embed query with `bge-small-en-v1.5`; retrieve top-30 candidates from ChromaDB by cosine similarity; apply cutoff $\tau_{\text{sem}} = 0.40$.

RRF Fusion. The two ranked lists are merged via Reciprocal Rank Fusion (RRF). For a document d, the fused score is:

$$\text{score}_{\text{RRF}}(d) = \sum_{i \in \{\text{BM25, dense}\}} \frac{1}{k + \text{rank}_i(d)}, \qquad k = 60 \tag{1}$$

where $\text{rank}_i(d)$ is the rank of d in list i. The constant k=60 dampens the winner-take-all effect of high ranks. A document absent from a list contributes zero ($\text{rank}_i(d) = \infty$).

Why hybrid + RRF? BM25 excels at exact-match recall (technical terms, arXiv IDs) while dense retrieval

captures paraphrase and semantic similarity. Neither alone is sufficient for academic text. RRF was preferred over learned fusion weights because it requires no training data, is parameter-free beyond k, and consistently performs well across domains.

Cross-Encoder Reranking *(optional).* The top-30 RRF-fused candidates are reranked by:

`cross-encoder/ms-marco-MiniLM-L-6-v2`

Unlike the bi-encoder, a cross-encoder attends jointly to the query and each passage, producing a calibrated relevance score. **Why this model?** The MiniLM-L-6 variant (80 MB) was chosen for its favourable accuracy-to-latency ratio; larger cross-encoders improve ranking but add seconds of latency per query. The top-5 reranked passages are forwarded to the Reflector.

Hard cutoffs. Thresholds $\tau_{\text{BM25}} = 1.0$ and $\tau_{\text{sem}} = 0.40$ discard irrelevant passages before fusion. This is the primary architectural defence against hallucinated citations: the Synthesizer can only cite passages that passed relevance filtering.

Multi-query deduplication. `retrieve_multi()` runs retrieval for each sub-query and merges results by `chunk_id`, retaining the highest RRF score for any chunk matched across multiple sub-queries.

3.5. Stage 3: Reflector

An iterative evidence-quality loop assesses whether the accumulated evidence is sufficient to answer the question. If not, it generates refined queries for another retrieval round, capped at three rounds (Algorithm 1).

Algorithm 1 Reflector Loop (`Agent.py`)

Require: question q, retriever R, max rounds T=3
queries ← Planner(q); $C \leftarrow \emptyset$
for $t = 1, \ldots, T$ **do**
  $C \leftarrow C \cup R$.`retrieve_multi`(queries)
  (ok, refined) ← LLM(q, `summarize`(C), t)
  if ok **or** refined = $\emptyset$ **then**
    break
  end if
  queries ← refined
end for
return C

Evidence sufficiency is judged on a compact summary (at most 6 chunks, each truncated to 200 chars) to stay within the LLM context budget. **Why a reflector?** Single-round retrieval frequently misses complementary evidence that requires different query formulations. The reflector allows the system to self-diagnose insufficient evidence and issue targeted follow-up queries, at the cost of additional LLM calls.

3.6. Stage 4: Synthesizer

Generates the final grounded answer from accumulated chunks using a structured LLM prompt enforcing a **three-tier claim discipline** (Table 1). At most 10 chunks (600 chars each) are passed, tagged `[PRIMARY]` (word-overlap with question ≥10%) or `[CONTEXT]`. Citations are restricted to papers in the retrieved chunk set; the prompt explicitly forbids citing any other source. Post-processing strips conversational preamble and normalises whitespace.

Table 1: Three-tier claim discipline enforced by the Synthesizer.

Tier	Label	Rule
A	Supported	Backed by a retrieved passage; must cite `[arxiv:ID]`
B	Background	General knowledge; no citation required
C	Uncertain	Not in evidence; flag in *Uncertainty notes*

Why a three-tier discipline? Without explicit tiers, LLMs generate fluent but unsupported statements with equal confidence to cited facts. Forcing the model to declare uncertainty for unsupported claims and background knowledge for general facts reduces hallucination while preserving useful contextual information in the answer.

3.7. Stage 5: Verifier

Performs deterministic, LLM-free citation grounding checks via lexical word overlap. For each cited `arxiv_id` in the answer:

1. If the ID is absent from the retrieved chunk set ⇒ marked **hallucinated**.
2. Compute word-overlap between all citing sentences and the source passage:

$$\text{ov}(S, P) = \frac{|W_S \cap W_P|}{|W_S|}$$

3. ov ≥ 0.20 ⇒ **verified**; else ⇒ **unverified**.

Two scalar metrics are derived:

$$F = \frac{|\text{verified}|}{|\text{cited}|}, \qquad P_c = \frac{|\text{cited}| - |\text{hallucinated}|}{|\text{cited}|}$$

where F is faithfulness and P_c is citation precision.

Why lexical overlap? An LLM-based verifier would add a sixth inference call per answer, approximately doubling marginal inference cost. Lexical overlap is deterministic, requires no model inference, executes in milliseconds, and is interpretable. Its primary weakness—brittleness on paraphrased claims—is documented as Failure Mode 3.

3.8. LLM Client (`llm_client.py`)

A unified multi-provider client uses the OpenAI-compatible chat completion format with automatic fallback in priority order:

$$\text{Groq} \rightarrow \text{Ollama} \rightarrow \text{OpenRouter} \rightarrow \text{OpenAI}$$

API keys are loaded from `.env`. Temperature is fixed at 0.1 for near-deterministic outputs. The client includes retry logic with exponential backoff and rate-limit handling.

Why this provider stack? Groq provides the lowest inference latency for the Llama 3.3 70B model class via a free API tier. Ollama enables fully local, offline execution with no API key, making the system deployable without any external dependency. OpenRouter and

OpenAI serve as fallbacks for model availability. The unified OpenAI-compatible interface means swapping providers requires no code changes, only a key rotation in `.env`.

4. Evaluation Setup

4.1. Question Set

Thirty questions from `eval/questions.jsonl` are divided equally across three types: **Factoid** (f01–f10, single-answer definitions), **Comparative** (c01–c10, two-method comparisons), and **Survey** (s01–s10, open-ended multi-faceted queries). This stratification allows per-type analysis even when aggregate metrics are the primary report.

4.2. Metrics

LLM-as-judge (1–5): Accuracy, Completeness, and Coherence are scored independently per answer by a separate judge LLM call (`evaluate.py`).

Faithfulness: F from the Verifier.

Citation Precision: P_c from the Verifier.

Efficiency: average wall-clock latency (seconds) and average LLM tool calls per question.

4.3. Ablation Configurations

Seven configurations isolate individual component contributions (Table 2). The `baseline` performs single-pass retrieval and synthesis with no agentic stages. Each ablation disables exactly one component relative to `full_agent`.

Table 2: Ablation matrix. ✓ = active; blank = disabled.

Configuration	Plan	Hyb	Rank	Refl	Ver
`full_agent`	✓	✓	✓	✓	✓
`baseline`					
`no_planner`		✓	✓	✓	✓
`no_reranker`	✓	✓		✓	✓
`no_reflector`	✓	✓	✓		✓
`no_hybrid`	✓		✓	✓	✓
`no_verifier`	✓	✓	✓	✓	

5. Results and Interpretation

Table 3 reports aggregated results across all 30 questions.

Table 3: Ablation results (n=30). Judge scores on 1–5 scale. **Bold** = best per column.

Config	Acc	Comp	Coh	Faith	Lat(s)	Calls
`full_agent`	3.733	**3.233**	**4.500**	0.985	30.27	3.9
`baseline`	3.767	3.033	4.267	**1.000**	**18.75**	**1.0**
`no_planner`	3.733	3.067	4.200	**1.000**	21.99	1.2
`no_reranker`	3.633	3.167	3.967	0.972	35.04	3.7
`no_reflector`	3.733	3.200	4.133	0.957	40.23	3.7
`no_hybrid`	3.733	3.133	4.467	0.942	28.42	3.8
`no_verifier`	**3.867**	3.200	4.267	**1.000**	31.18	3.7

5.1. Does the Full Agent Improve Over Baseline?

On accuracy, no; on coherence and completeness, yes. The `baseline` achieves the second-highest Accuracy (3.767 vs. 3.733) at 1.6× lower latency (18.75 s vs. 30.27 s) with only one LLM call. However, the `full_agent` achieves substantially higher Coherence (4.500 vs. 4.267) and Completeness (3.233 vs. 3.033). This reveals a fundamental trade-off: agentic scaffolding improves answer structure and coverage at the cost of speed and marginal accuracy reduction.

Caveat. Aggregate scores mask per-type variation. Survey questions informally benefit more from the reflector than factoid questions, consistent with iterative retrieval helping open-ended queries. A formal per-type breakdown was not available for all configurations at time of writing and constitutes an important limitation.

5.2. Is the Reranker Essential?

Yes, and it is the most impactful component. Removing the reranker (`no_reranker`) produces the lowest Accuracy (3.633) and Coherence (3.967) across all configurations—the largest single drop in the study. This confirms that cross-encoder reranking provides substantial value beyond RRF fusion alone by better distinguishing relevant from semi-relevant passages in technical academic text. The ablation result validates the additional latency of reranking (35.04 s vs. 28.42 s without hybrid).

5.3. Does the Planner Help?

Marginally, with a latency benefit. Removing the planner (`no_planner`) reduces latency by 27% (21.99 s vs. 30.27 s) with no accuracy change (3.733 vs. 3.733). The `no_planner` configuration approaches baseline latency (21.99 s vs. 18.75 s) while retaining hybrid retrieval and reflection, making it a practical choice when speed is prioritised. The planner's primary measurable cost is the additional LLM call for decomposition.

5.4. Does the Reflector Improve Coverage?

Yes, but with a counterintuitive latency anomaly. The `no_reflector` configuration records *higher* latency (40.23 s) than `full_agent` (30.27 s) despite having one fewer stage. We hypothesise that the reflector's early-termination logic frequently short-circuits the loop before all three rounds complete, reducing total LLM call time. Without it, all retrieval and synthesis calls proceed without interruption. Completeness drops marginally (3.200 vs. 3.233) without the reflector, confirming it contributes to coverage.

5.5. Does the Citation Verifier Add Value?

Not for accuracy; it may actively harm it. `no_verifier` achieves the *highest* Accuracy (3.867) in the entire study, with perfect Faithfulness and Citation Precision. This counter-intuitive result, corroborated by Failure Mode 3, suggests the lexical overlap verifier is overly conservative, flagging semantically valid paraphrased citations as unverified and thereby degrading the final answer quality signal used by the judge. The verifier currently provides interpretability but no measurable accuracy benefit.

5.6. Component Importance Ranking

Ranked by measured impact on Accuracy:

(1) **Reranker**—largest drop when removed (−0.100);

(2) **Verifier**—negative impact; removal *improves* accuracy (+0.134);
(3) **Planner / Reflector / Hybrid**—marginal impact ($|\Delta| \leq 0.034$);
(4) **Baseline**—best accuracy/latency trade-off overall.

6. Failure Modes

FM-1: Planner Over-Decomposition on Factoid Queries

Example (f03 — "What is the ReAct prompting framework?"). The planner generates four sub-questions: definition, components, tasks, and comparison with CoT. Each retrieves overlapping passages from the same source paper. The synthesizer produces a longer, less focused answer (Coherence 3.2 vs. 3.9 for baseline on this question).

Root cause. Uniform decomposition is applied regardless of question type. Atomic factoid questions do not benefit from multi-query expansion; the rule-based fallback triggers only for very short queries and does not fully compensate.

Fix. Classify queries (factoid / comparative / survey) before planning. Bypass decomposition for factoid; apply two-part decomposition for comparative.

FM-2: Reflector Topic Drift on Survey Queries

Example (s07 — "Agent memory architectures 2024–2025"). Round 1 retrieves episodic memory passages; reflector judges insufficient. Round 2 targets "parametric memory editing," causing continual learning papers to dominate. Round 3 incorrectly marks evidence sufficient; retrieval-based memory systems are under-represented in the final answer despite being well covered in the corpus.

Root cause. Refined queries target perceived gaps without preserving coverage of already-retrieved facets.

Fix. Maintain a facet coverage tracker across rounds. Constrain refined queries to not reduce existing facet representation below a minimum threshold and enforce semantic proximity to the original question.

FM-3: Verifier False Negatives on Paraphrased Claims

Example.
Source: "a hierarchical planning mechanism decomposes tasks into subtasks delegated to sub-agents."
Answer: "the framework employs a multi-level delegation strategy where macro-objectives are assigned to purpose-built sub-agents `[arxiv:ID]`."
Word overlap: $1/20 = 0.05 < 0.20$; citation marked **unverified** despite clear semantic support. This failure directly explains why `no_verifier` achieves the highest accuracy in Table 3.

Root cause. Lexical overlap is brittle on paraphrased claims, which are the norm in academic writing where authors routinely rephrase prior work.

Fix. Augment with a lightweight NLI model (`nli-deberta-v3-small`): classify each (citing sentence, source passage) pair as entailment / neutral / contradiction. Entailment counts as verified; the check remains LLM-free and deterministic.

7. Future Work

1. **Adaptive planning.** Classify query type before decomposition; bypass the planner for factoid and apply targeted decomposition for survey queries.
2. **Scope-preserving reflection.** Track facet coverage across rounds; constrain refined queries to maintain existing facet representation above a threshold.
3. **Semantic citation verification.** Replace lexical overlap with NLI-based entailment to eliminate paraphrase false negatives while remaining LLM-free.
4. **Domain-specific reranking.** Fine-tune the cross-encoder on academic passage pairs rather than relying on MS MARCO web-text training.
5. **Per-type evaluation.** Report all metrics separately for factoid, comparative, and survey questions to enable targeted component selection.
6. **Efficiency optimisation.** Early-stopping for the reflector; embedding caching for repeated sub-queries; batch inference for reranking.

8. Conclusion

We presented an end-to-end agentic RAG system and a seven-way ablation study evaluating whether agentic scaffolding improves over a strong single-pass baseline on a bounded academic corpus. The honest answer is **not uniformly.** The baseline achieves higher accuracy at substantially lower latency; the full agent achieves superior coherence and completeness. The reranker is the single most impactful component; its removal causes the largest accuracy drop. Surprisingly, the citation verifier shows no accuracy benefit and may actively harm performance by flagging valid paraphrased citations as unverified.

Three systematic failure modes—over-decomposition, topic drift, and lexical verifier brittleness—demonstrate that the agentic loop requires targeted improvements rather than general scaling. The modular `Agent.py` architecture and ablation suite provide a reproducible foundation for the proposed fixes. These findings are specific to this corpus size, domain, and question set; generalisation to larger or more diverse corpora requires additional evaluation.